不同搜索树的比较

2026 年 10 月 19 日

Chapter 1

引言

1.1 实验目的

树形结构，是计算机科学中最常用的数据结构之一。它们在数据库、文件系统、编译器等领域都有广泛的应用。不同的树结构在不同的应用场景下有不同的性能表现。本实验旨在实现并比较几种常见的搜索树，包括二叉搜索树（BST）、AVL 树、红黑树（RBT）、伸展树（Splay Tree）和 B+ 树（BPT），以了解它们在插入、删除操作上的性能差异。

1.2 实验环境

- 编程语言：C
- 编译器：gcc
- 在算法实现上，我们都使用了迭代的方式，以尽可能控制变量。

1.3 数据参数

本实验当中，我们以 1 N 的顺序插入树中，删除时，分别统计顺序删除、逆序删除以及随机删除的用时。考虑到使用了随机序列，我们在这里将每次插入、删除执行都若干次（实验中是 10 次，详情见 README.md）以减少偶然性。在随机序列生成时，我们使用了 Fisher-Yates 洗牌，具体的算法流程为：

```
function RandomOrder(n)
  initial list as (1,2,....,n)

   //索引是0-based
   for (i=n-1,i>=0;i--)
      get a random number k
      l = k % i+1
      swap list[i] and list[l]
```

假设随机数 k 的生成是完全随机的。对任意 (1-n) 的随机序列 $(a_1, a_2, ..., a_n)$，从末尾开始，每次选中 a_i 的概率是 $\frac{1}{i}$。序列生成的概率为 $p = \Pi_{i=n}^{1} \frac{1}{i} = \frac{1}{n!}$ 。这说明这个算法生成的序列是随机的。

Chapter 2

数据结构 & 算法描述

本实验当中用到的数据结构有：

- 二叉搜索树 (BST)
- AVL
- 红黑树 (RBT)
- 伸展树 (Splay Tree)
- B+ 树 (BPT)

我们将会依次描述其实现方式，给出相应算法的描述（伪代码）。

2.1 二叉搜索树

BST 维护左子树键值小于根、右子树键值大于根的性质。普通 BST 不主动调整树高，有序插入时可能退化为高度为 N 的链。删除时通过额外指针记录当前结点的父结点，无需使用栈，额外空间复杂度为 $O(1)$。

插入

```
function insertBST(T,key)
   current = T

   // 空树时，新结点直接成为根
   if (current == NULL)
      return newNode(key)

   // 按键值大小沿树向下查找插入位置
   while (True)
      if (key < current->data)
         // 找到空左孩子后插入；否则继续向左搜索
         if (current->left == NULL)
            current->left = newNode(key)
            break
         current = current->left

      else if (key > current->data)
         // 找到空右孩子后插入；否则继续向右搜索
         if (current->left == NULL)
            current->left = newNode(key)
            break
         current = current->left

```

```
        else
            // 键值已存在，不重复插入
            break
```

删除

```
function deletionBST(T,key)
    parent = NULL
    current = T

    // 空树无需删除
    if (T == NULL)
        return T

    // 查找目标结点，同时记录其父结点以便后续摘除
    while (current != NULL and current->val != key)
        parent = current
        if (key < current->val)
            current = current->left
        else
            current = current->right

    // 未找到目标键，树保持不变
    if (current == NULL)
        return T

    // 双孩子结点用右子树中的最小结点替换
    if (current has two non-NULL children)
        successorParent = current
        successor = current->right
        while (successor->left != NULL)
            successorParent = successor
            successor = successor->left
        current->val = successor->val
        parent = successorParent
        current = successor

    // 至多一个孩子：让父结点直接接上该孩子
    child = the non-NULL child of current, if any
    if (parent == NULL)
        T = child
    else if (parent->left == current)
        parent->left = child
    else
        parent->right = child

    free(current)
    return T
```

2.2 AVL

AVL 使用平衡因子 (balance factor)，来维持树的平衡。记树 T 的高度为 H_T，空树高度为-1。每个结点的平衡因子 $bf = H_{T->left} - H_{T->right}$。对于一棵 AVL 树，其每个节点的左右子树的高度差小于等于 1，即 $bf \in \{-1, 0, 1\}$。对树的插入删除都可能会破坏这一性质，因此，我们需要旋转这种操作来维护这种性质。

2.2.1 旋转

旋转，分为左旋和右旋两种操作，由于两者是镜面对称的，因此我们在这里以左旋为例。

```
function LeftRotate(root)
    // 将右孩子提升为新根，并把其左子树转接给原根
    pivot = root->right
    root->right = pivot->left
```

```
    pivot->left = root

    // 子树结构改变后，先更新较低结点再更新新根
    Update the height and balance factor of root and pivot
    return pivot
```

右旋与左旋对称；旋转后返回新的子树根。旋转操作在接下来几种二叉树当中均会使用，后续就不再赘述了。

2.2.2 插入

先根据 BST 的插入方式找到插入位置，然后将新结点插入到该位置。插入后如果破坏了 AVL 树的性质，则需要进行旋转操作来恢复平衡。

```
function AVLInsert(node, key)
    // 保存搜索路径，便于插入后自底向上更新和再平衡
    path = empty stack
    current = node

    Search for the insertion position as in a binary search tree
    and push the node passing by onto the stack

    child = CreateNode(key)
    while (path is not empty)
        current = Pop path
        // 将已处理的子树接回当前结点，再更新其平衡信息
        Attach child to current on the side containing key
        Update the height and balance factor of current

        if (current is unbalanced)
            // 先处理内侧子树旋转，形成单旋可修复的形态
            if (left subtree is higher)
                if (key > current->left->key)
                    current->left = LeftRotate(current->left)
                child = RightRotate(current)
            else
                if (key < current->right->key)
                    current->right = RightRotate(current->right)
                child = LeftRotate(current)
        else
            child = current

    return child
```

2.2.3 删除

类似的，先根据 BST 的删除方式找到删除位置，然后将该结点删除。相较于插入，删除之后可能需要更多次的旋转以维持平衡。

```
function AVLDelete(node, key)
    // 保存搜索路径及方向，供删除后逐层接回子树
    path = empty stack
    current = node

    Search for the deletion position as in a binary search tree
    and push the node passing by onto the stack

    if (current == NULL)
        return node

    if (current has two children)
        // 用后继替换目标结点，实际摘除后继并从其位置开始回溯
        Push current and the right-child direction onto path
        successor = current->right
        while (successor->left != NULL)
```

```
        Push successor and the left-child direction onto path
        successor = successor->left
    current->key = successor->key
    child = successor->right
    Free successor
else
    child = current's non-NULL child, if any
    Free current

while (path is not empty)
    (current, direction) = Pop path
    // 接回删除后的子树，并向根方向更新高度和平衡因子
    Attach child to current in direction
    Update the height and balance factor of current

    if (current is unbalanced)
        // 根据较高子树的平衡因子选择单旋或先子树后根的双旋
        if (left subtree is higher)
            if (current->left balance factor < 0)
                current->left = LeftRotate(current->left)
            child = RightRotate(current)
        else
            if (current->right balance factor > 0)
                current->right = RightRotate(current->right)
            child = LeftRotate(current)
    else
        child = current

return child
```

2.3 红黑树

红黑树对每个结点标记颜色（红/黑），其中结点的颜色需满足以下几条性质：

- 根的颜色为黑色。
- 不能出现连续两个红色结点。
- 每条分路（从根到空结点）的黑色结点数相同。
- 空结点视为黑色。

这个性质保证了最长路径（红黑交替）不超过最短路径（全黑）的两倍。

2.3.1 插入

这里，我们在每个结点当中保存了其父结点的地址，方便回溯、调整。

```
function RBTInsert(T, key)
    // 先按 BST 规则定位插入位置；新结点初始为红色
    Search for the insertion position as in a binary search tree
    If key already exists, return T
    Create a red node new and attach it to its parent
    If new is the root, color it black and return new
    // 仅当父结点为红色时，才需要修复连续红结点
    while (new is not the root and its parent is red)
        parent = new->parent
        grandparent = parent->parent
        uncle = the other child of grandparent

        if (uncle is red)
            // 叔结点为红：向上调整冲突，将修复范围提升到祖父结点
            Color parent and uncle black, and grandparent red
            new = grandparent
```

```
        else
            // 叔结点为黑：通过旋转和重新着色消除局部红红冲突
            if (parent is the left child of grandparent)
                if (new is the right child of parent)
                    new = parent
                    LeftRotate(parent)
                Color new's parent black and grandparent red
                RightRotate(grandparent)
            else
                if (new is the left child of parent)
                    new = parent
                    RightRotate(parent)
                Color new's parent black and grandparent red
                LeftRotate(grandparent)
            Stop adjusting

    Follow parent pointers to the root and color it black
    return T
```

2.3.2 删除

删除时将实际摘除结点的原颜色记录下来。若其为黑色，则通过调整替代结点及其兄弟结点，恢复各条路径的黑高；空结点视为黑色。

```
function RBTDelete(T, key)
    // y 是实际被移出原位置的结点；修复依据其原始颜色
    z = Search for key as in a binary search tree
    if (z == NULL)
        return T

    if (z has two non-NULL children)
        y = findmin(z->right)
        z->data = y->data
        z = y

    //删除结点单孩子，替代后变色
    if (z has one non-NULL child)
        replace z with its child
        Color new node red
        if (z == T) update root
        return T

    originalColor = z->color

    zparent = z->parent
    Free z
    z = NULL

    //删除无孩子的黑节点
    if (originalColor == BLACK)
        // 黑结点被移除会造成黑高缺失，逐层借色或旋转修复
        while (true)
            s = the other child of zparent

            //双黑遇到根或者是红节点，染黑（这里z为NULL是黑色）
            if (z is RED or z is root)
                z->color = BLACK
                break

            //兄弟为黑色
            if (s->color == BLACK)
                if (s = zparent->left/right)
                    //兄弟、侄子、父亲呈LL/RR
                    //颜色变换：侄子<-兄弟<-父亲<-BLACK
                    //对应方式旋转，类似于AVL
                    if (s->left/right ->color == RED)
                        s->left/right ->color = s->color
                        s->color = zparent->color
```

```
                    zparent->color = BLACK
                    RightRotate/LeftRotate(zparent)
                    break

                //兄弟、侄子、父亲呈LR/RL
                //颜色变换：侄子<-父亲<-BLACK
                //对应方式旋转，类似于AVL
                else if (s->right/left ->color == RED)
                    s->right/left ->color = zparent->color
                    zparent->color = BLACK
                    LeftRotate/RightRotate(s)
                    RightRotate/LeftRotate(zparent)
                    break

                //侄子双黑，兄弟变色，双黑上移
                else
                    s->color = RED
                    z = zparent
                    zparent = z->parent //约定：root的parent为NULL
                    continue

        //兄弟红色，兄父变色，朝双黑旋转，继续调整
        if (s->color == RED)
            s->color = BLACK
            zparent->color = RED

            if (z is the left/right child of zparent)
                RightRotate/LeftRotate(zparent)

    If x is not NULL, color it black
    If T is not NULL, follow parent pointers to the root
    return T
```

2.4 伸展树

Splay Tree 在每次访问后将目标节点（或最后访问节点）伸展到根，单次最坏复杂度为 $O(N)$，但任意序列的摊还复杂度为 $O(\log N)$。

在这里，Splay 使用 top-down 算法。临时头结点把访问路径拆为左、中、右三部分，循环执行 zig-zig 和链接，最后重新拼接。插入先伸展目标键，再把新节点置于根；删除先伸展目标，若存在则将左子树最大节点伸展到根并接回原右子树。这样不需要保留父节点信息，也不需要使用显式栈。

top-down 算法当中，重要操作就是 splay

```
function splay(T, key)
    if (T == NULL)
        return NULL

    // 临时头结点分别串接访问路径左侧和右侧的结点
    header.left = NULL
    header.right = NULL
    leftTree = header
    rightTree = header
    root = T

    while (True)
        if (key < root->val)
            if (root->left == NULL)
                break
            if (key < root->left->val)
                // 连续向左：先右旋，执行 zig-zig
                root = RightRotate(root)
                if (root->left == NULL)
                    break
            // 当前根归入右侧路径，继续在其左子树中查找
            rightTree->left = root
```

```
            rightTree = root
            root = root->left
        else if (key > root->val)
            if (root->right == NULL)
                break
            if (key > root->right->val)
                // 连续向右：先左旋，执行镜像的 zig-zig
                root = LeftRotate(root)
                if (root->right == NULL)
                    break
            // 当前根归入左侧路径，继续在其右子树中查找
            leftTree->right = root
            leftTree = root
            root = root->right
        else
            break

    // 将两侧临时链与最终根的子树重新拼接
    leftTree->right = root->left
    rightTree->left = root->right
    root->left = header.right
    root->right = header.left
    return root
```

插入

```
function insertSplay(T, key)
    if (T == NULL)
        return newNode(key)

    // 伸展后，若键不存在，根即为新键应插入位置的一侧边界
    T = splay(T, key)
    if (T->val == key)
        return T

    node = newNode(key)

    if (key < T->val)
        // 新结点成为根：接管原根的左子树，原根成为其右孩子
        node->left = T->left
        node->right = T
        T->left = NULL
    else
        // 镜像情况：接管原根的右子树，原根成为其左孩子
        node->right = T->right
        node->left = T
        T->right = NULL

    return node
```

删除

```
function delete(root, key):
    if (root == NULL)
        return NULL

    root = splay(root, key)

    // 不存在
    if (root.key != key)
        return root

    // 左子树为空时，右子树直接成为新根
    if (root.left == NULL)
        newRoot = root.right
    else
        // 将左子树最大结点伸展到根，再把原右子树接回
        newRoot = splay(root.left, key) // key 在左子树中最大
        newRoot.right = root.right   // 此时 newRoot.right 必为 NULL
    return newRoot
```

2.5 B+ 树

与其他的树不同的是，B+ 树是一种多叉的搜索树，所有的数据存储在叶子结点上，每个叶子结点的高度相同，同时其叶子结点之间通过链表相连，方便范围查询。其多叉的性质使得 B+ 树在磁盘存储上有更好的性能，可以显著减少磁盘 I/O 次数。对于一棵 order 为 M 的 B+ 树，其每个结点最多有 M 个子结点，至少有 $\lceil M/2 \rceil$ 个子结点。在实现方式上，我们使用结点的数组来存储其子结点的指针，同时使用一个数组来存储其关键字。

数据结构体定义如下：

```
typedef struct Bn
{
    int leaf; //判断是否为叶节点，便于分配链表指针
    int* data; //关键字数组
    int num; //当前关键字个数

    struct Bn** children; //子结点指针数组
    struct Bn* next; //叶子结点的链表指针，指向下一个叶子结点
    struct Bn* prev; //叶子结点的链表指针，指向上一个叶子结点
}BNode;
```

插入

```
//树order为M
function insertBPT(T,key)
    // 搜索路径用于在叶子分裂后逐层向父结点插入分隔键
    path = empty stack
    Search for the insertion position as in a search tree
    and push the node passing by onto the stack

    I = key
    // C1、C2 表示待插入分隔键两侧的子树；叶子插入时均为空
    C1 = NULL
    C2 = NULL
    while (True)
        if (path is empty)
            // 分裂一路传播到根时，创建新根
            root = newNode()
            root->data[0] = I
            root->children[0] = C1
            root->children[1] = C2
            if (C1 != NULL && C2 != NULL)
                root->leaf = true
            l=else
                root->leaf = false
            T = root
            break

        current = Pop path

        // 将待插入键及其子树指针插入当前结点
        insert I into current's data array

        if (C1 != NULL && C2 !=NULL)
            insert C1&C2 into current's children array in the corresponding position

        if (current->num > M)
            // 结点溢出则分裂，并把分隔键和两棵子树继续向上推送
            split current into two node: A & B

            if (current is leaf)
                // 叶子分裂后维护双向叶链表
                A->next = B
                B->prev = A
                if (current->prev != NULL)
                    current->prev->next = A
                else
                    update the ptr pointing to the head of the linked list
```

```
            B->next = current->next

        free(current)

        // B 的首键作为父结点的分隔键继续插入
        I = the first key of B
        C1 = A
        C2 = B

    else
        break

return T
```

删除

```
function deleteBPT(T,key)
    // 沿内部结点定位目标叶子，并保存祖先路径以便向上修复
    if (T->root == NULL)
        return NULL

    path = empty stack
    current = T->root
    while (current is not a leaf)
        Find childIndex such that key belongs to current->children[childIndex]
        Push (current, childIndex) onto path
        current = current->children[childIndex]

    Find key in current->data
    if (key is not found)
        return T

    // 从叶子删除目标键；若最小键改变，更新祖先分隔键
    Remove key from current->data and decrease current->num
    if (the deleted key was the first key and current->num > 0)
        Update the affected separator key in its ancestors

    minKeys = floor(T->order / 2)
    while (current is not the root and current->num < minKeys)
        // 自底向上修复下溢：优先向兄弟借键，否则合并
        (parent, childIndex) = Pop path
        left = parent->children[childIndex - 1], if childIndex > 0
        right = parent->children[childIndex + 1], if childIndex < parent->num

        if (current is a leaf)
            if (left/right exist and have at least minKeys)
                // 叶子向相邻兄弟借一个键，并同步父结点分隔键
                Move left's/right's largest/smallest key to the front of current->data
                Update parent->data
                Update affected ancestor separators if the subtree minimum changed
                break
            else if (left/right exist but have less than minKeys)
                // 叶子无法借键时合并，并保持叶子链表连续
                merge current and left/right
                update linked list pointers
                update parent->data and parent->children
                Free current

        else
            if (left/right exist and have at least minKeys)
                // 内部结点借键时，同时旋转父分隔键及对应子树指针
                Move parent->data[childIndex - 1]/[childIndex] to the front/end of current->data
                Move left's/right's last/first child to the front/end of current->children
                Move left's/right's largest/first key up to parent->data[childIndex - 1]/[childIndex]
                break
            else if (left/right exist but have less than minKeys)
                // 合并内部结点，并将父分隔键下移到合并后的结点
                merge current and left/right
                Remove parent->data[childIndex - 1]/[childIndex] and parent->children[childIndex]/[childIndex + 1]
                Free current
```

```
55
56          current = parent
57
58      // 根结点可少于最小占用量；空内部根收缩为唯一孩子
59      if (T->root is an internal node and T->root->num == 0)
60          oldRoot = T->root
61          T->root = oldRoot->children[0]
62          Free oldRoot
63
64      if (T->root is a leaf and T->root->num == 0)
65          Free T->root
66          T->root = NULL
67          T->head = NULL
68          return T
69
70      T->head = the leftmost leaf under T->root
71      return T
```

Chapter 3

实验结果

我们将实验当中各个树用时与数据量的关系作图，分为线性坐标轴与对数坐标轴。图表当中的用时是摊还用时，即若干次操作的总用时除以操作的数量。

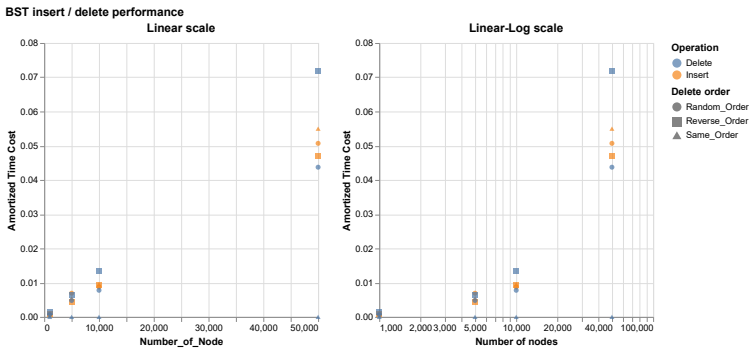


图 3.1: BST 实验结果

对于普通 BST，当插入数据是有序时，树会退化为链表。此时如果按插入顺序删除，每次删除的结点正好在根，这使得删除只需要消耗 $O(1)$ 的时间。而若是按逆序删除，则每次删除的结点都在叶子结点，删除需要消耗 $O(N)$ 的时间。实验当中顺序删除的用时几乎为零，而逆序删除的用时呈现线性增长。且耗时远大于接下来的几种平衡二叉树。

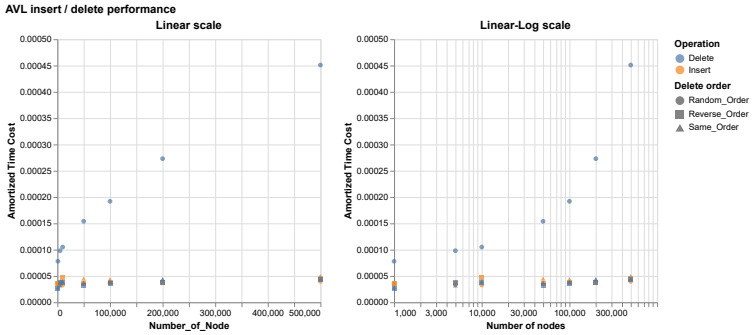


图 3.2: AVL 实验结果

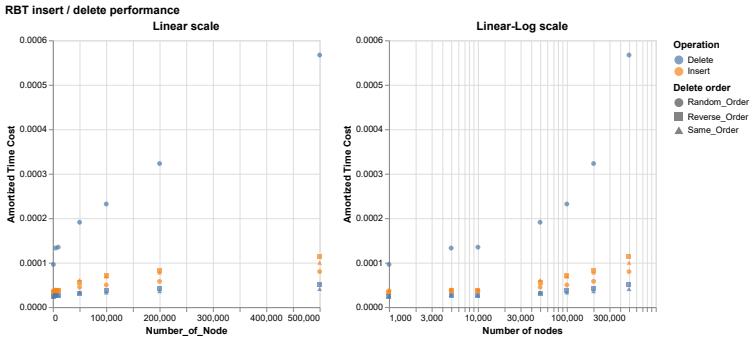


图 3.3: RBT 实验结果

AVL 和 RBT 都是平衡二叉树，插入和删除的时间复杂度均为 $O(\log N)$。这点在实验结果中得到了验证，随着数据量的增加，AVL 和 RBT 的用时呈现对数增长。但是 AVL 的平衡相较于 RBT 的平衡更加严格，因此 AVL 的插入和删除操作需要更多的旋转操作，导致其用时略高于 RBT。这点在我们的实验图像当中得到了体现，AVL 的曲线在 RBT 曲线之上。这说明 RBT 在综合性能上优于 AVL，尤其是在插入和删除操作频繁的场景下。

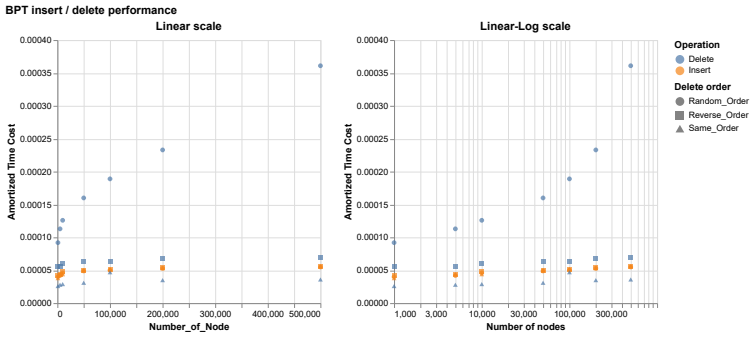


图 3.4: BPT 实验结果

BPT 是一种多叉平衡树，适用于磁盘存储和大数据量的场景。由于 BPT 的多叉性质，它在插入和删除操作中需要进行更多的节点分裂和合并操作，这使得其在小数据量下的性能不如 AVL 和 RBT。然而，随着数据量的增加，BPT 的性能优势逐渐显现出来，尤其是在大数据量下，BPT 的用时明显低于 AVL 和 RBT。

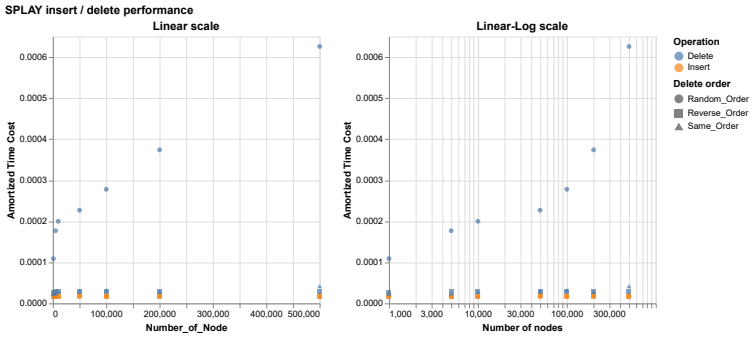


图 3.5: SPLAY 实验结果

Splay Tree 在每次访问后将目标节点伸展到根，这使得它在访问局部性较强的场景下表现出色。在我们的实验中，Splay Tree 的用时在有序插入和删除的情况下表现优异，尤其是在顺序删除的场景下，其用时几乎为常数。

Chapter 4

总结

本次实验，我们实现了五种常见的搜索树，并对它们在插入和删除操作上的性能进行了比较。实验结果表明：

- 普通 BST 在有序数据下会退化为链表，导致插入和删除操作的性能大幅下降。
- AVL 和 RBT 作为平衡二叉树，能够保持较好的性能，其中 RBT 在综合性能上优于 AVL。
- BPT 在大数据量下表现出色，适用于磁盘存储和大数据量的场景。
- Splay Tree 在访问局部性较强的场景下表现优异，尤其是在顺序删除的情况下，其用时几乎为常数。

综上所述，选择合适的搜索树结构对于不同的应用场景至关重要。在实际应用中，应根据数据的特点和操作的频率来选择最适合的搜索树，以获得最佳的性能表现。